Введение в паттерны проектирования GoF

(GoF - Gang of Four/ Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides)

Термины и определения

Паттерн проектирования[1] или шаблон (*англ. design pattern*) в разработке программного обеспечения — это повторяемая архитектурная конструкция, представляющая собой решение проблемы проектирования в рамках некоторого часто возникающего контекста

«любой паттерн описывает задачу, которая снова и снова возникает в нашей работе, а также принцип ее решения, причем таким образом, что это решение можно потом использовать миллион раз, и при этом никакие две реализации не будут полностью одинаковыми»

(Christopher Alexander, Sara Ishikawa, Murray Silverstein, Max Jacobson, Ingrid Fiksdahl-King, and Shlomo Angel. A Pattern Language. Oxford University Press, New York, 1977)

Описание паттерна

Имя
Описывает проблему проектирования в одном-двух словах.

Задача.
Описание того, когда следует применять паттерн. Описание объясняет задачу и ее контекст.
•может описываться конкретная проблема проектирования, например способ представления алгоритмов в виде объектов.
•могут быть отмечены структуры классов или объектов.
•может включать перечень условий, при выполнении которых имеет смысл применять данный паттерн.

Решение.
Описание элементов проектирования, отношений между ними, их обязанностей и взаимодействий между ними.
Приводится абстрактное описание задачи проектирования и ее возможного решения с помощью некоего обобщенного сочетания элементов (классов и объектов).

Результаты
Следствия применения паттерна, его вероятные плюсы и минусы.
- подробности реализации
- баланс затрат времени и памяти
- влияние на степень гибкости, расширяемости и переносимости системы

Термины и определения

Технически, ***паттерны (шаблоны)*** проектирования - это абстрактные примеры (приемы) правильного использования небольшого числа комбинаций простейших техник ООП (объектно-ориентированного программирования).

Паттерны проектирования - это простые примеры, показывающие правильные способы организации взаимодействий между *классами* или *объектами*.

Паттерны описывают:

- правильные способы формирования внутреннего состояния (полей) и поведения (методов) объекта или класса;
- правильные способы создания объекта (через вызов конструктора или другим способом);
- правильные способы объединения объектов в группы;
- правильные способы организации информационных потоков (вызовов методов и очередности вызовов) позволяющих наладить гармоничное взаимодействие между объектами и группами этих объектов в объектно-ориентированных системах.

Под **паттернами проектирования**[2] понимается описание взаимодействия объектов и классов, адаптированных для решения общей задачи проектирования в конкретном контексте.

Паттерн проектирования именует, абстрагирует и идентифицирует ключевые аспекты структуры общего решения, которые позволяют применить его для создания повторно используемого решения.

Паттерн выделяет участвующие классы и экземпляры, их роли и отношения, а также распределение обязанностей. При описании каждого паттерна внимание акцентируется на конкретной задаче объектно-ориентированного проектирования.

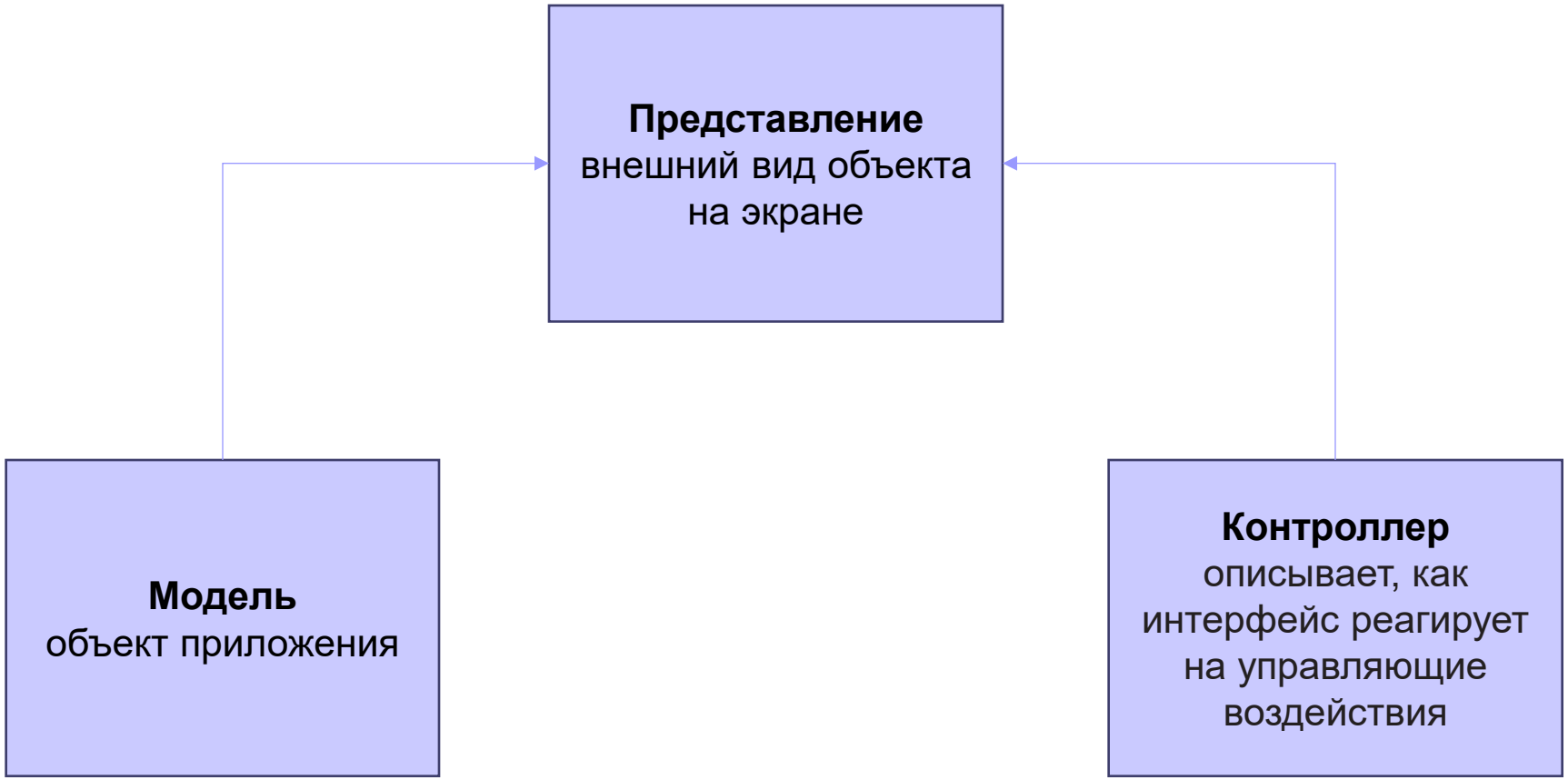
В *формулировке* паттерна анализируется, когда следует применять паттерн, можно ли его использовать с учетом других проектных ограничений, каковы будут последствия применения метода.

Поскольку любой проект в конечном итоге предстоит реализовывать, в состав паттерна включается *пример кода* на языке, иллюстрирующий реализацию.

Паттерны проектирования в схеме MVC

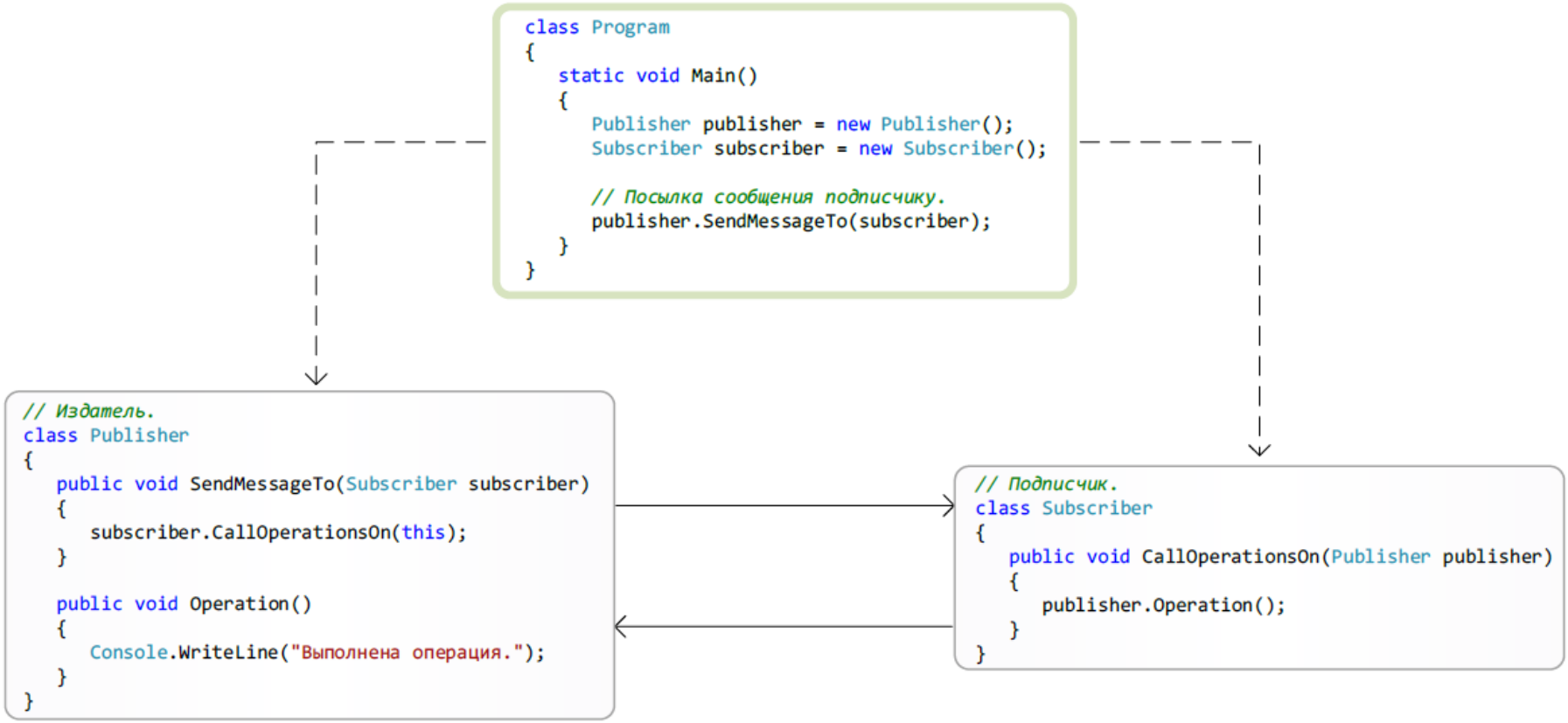
Для построения интерфейсов пользователя широко применяется тройка классов *модель/представление/контроллер* (Model/View/Controller — **MVC**)

MVC состоит из объектов трех видов:



Паттерны проектирования в схеме MVC

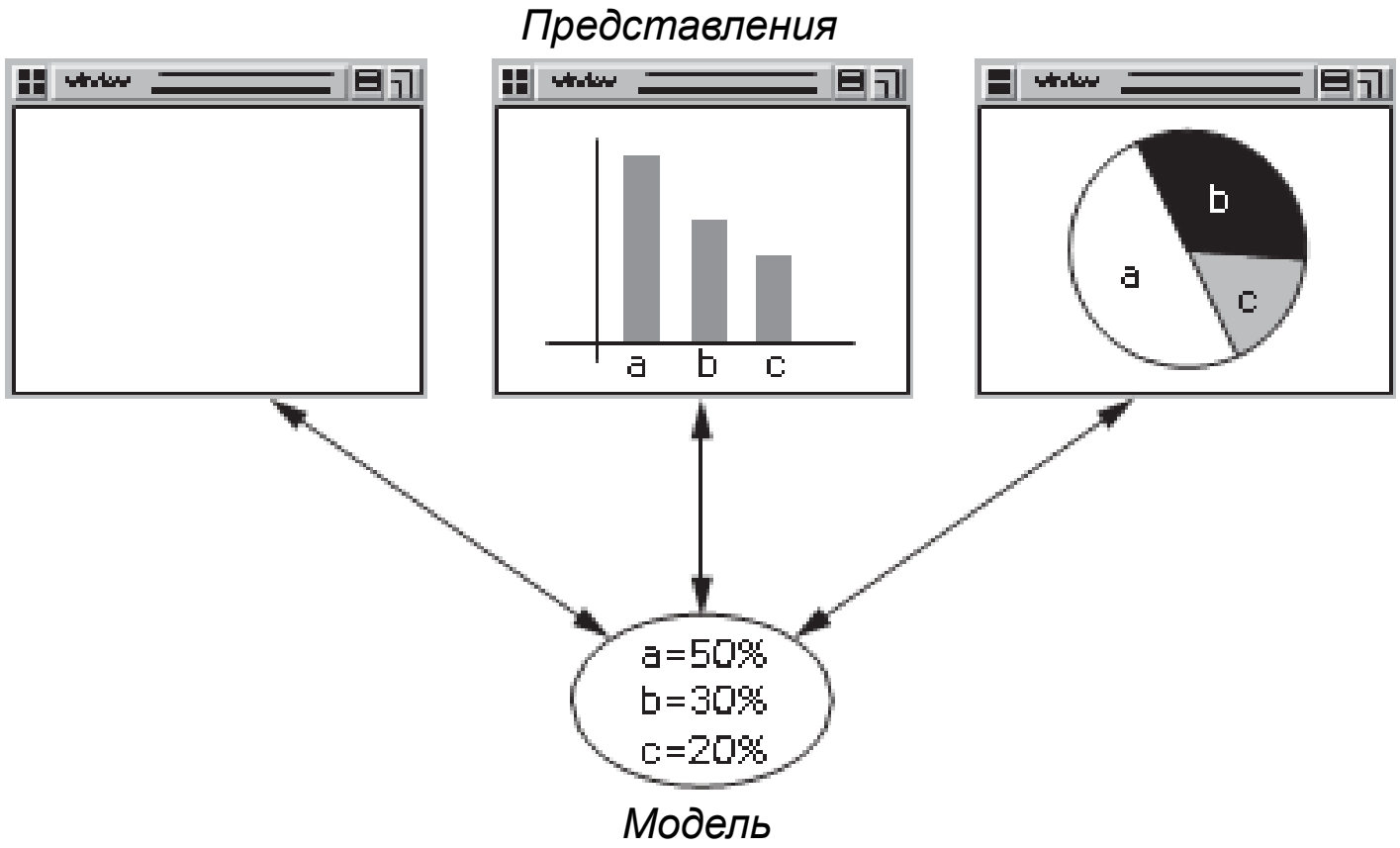
MVC отделяет *представление* от *модели*, устанавливая между ними протокол взаимодействия *«подписка/уведомление»*.



Паттерны проектирования в схеме MVC

Представление должно гарантировать, что внешнее представление отражает состояние *модели*. При каждом изменении внутренних данных *модель* уведомляет все зависящие от нее *представления*, в результате чего *представление* обновляет себя.

Такой подход позволяет присоединить к одной *модели* несколько *представлений*, обеспечив тем самым различные представления. Можно создать новое *представление*, не переписывая *модель*.



Паттерны проектирования в схеме MVC

Этот принцип применим и к более общей задаче:

разделение объектов таким образом, что изменение одного отражается сразу на нескольких других, причем изменившийся объект не имеет информации о подробностях реализации других объектов.

Этот подход описывается паттерном проектирования **наблюдатель.**

Такой подход позволяет обращаться с составным представлением, как с любым из его компонентов. *Например*, в ситуации, когда необходимо иметь возможность группировать объекты и рассматривать группу как отдельный объект.

Подобное решение описывается паттерном **компоновщик**.

Он позволяет создавать иерархию классов, в которой некоторые подклассы определяют примитивные объекты, а другие - составные объекты, группирующие примитивы в более сложные структуры.

MVC позволяет также изменять реакцию представления на действия пользователя. При этом визуальное воплощение остается прежним.

MVC инкапсулирует механизм определения реакции в объекте Controller.

Существует иерархия *классов контроллеров*, и это позволяет создать новый контроллер как вариант уже существующего.

Паттерны проектирования в схеме MVC

Представление пользуется экземпляром класса, производного от *Controller*, для реализации конкретной стратегии реагирования.

Чтобы реализовать иную стратегию, нужно подставить другой контроллер.

Можно заменить контроллер представления во время выполнения программы, изменив тем самым реакцию на действия пользователя.

Отношение *представление/контроллер* — это пример паттерна проектирования **стратегия**.

Стратегия — это объект, представляющий алгоритм.

Он будет полезен, когда необходимо статически или динамически подменить один алгоритм другим, если существует много разновидностей одного алгоритма или когда с алгоритмом связаны сложные структуры данных, которые нужно инкапсулировать.

Таким образом, основные отношения в схеме MVC описываются паттернами **наблюдатель, компоновщик и стратегия**

Классификация паттернов

Группы паттернов

Порождающие

1. **Abstract Factory** (Абстрактная Фабрика)
2. **Builder** (Строитель)
3. **Factory Method** (Фабричный Метод)
4. **Prototype** (Прототип)
5. **Singleton** (Одиночка)

Структурные

1. **Adapter** (Адаптер)
2. **Bridge** (Мост)
3. **Composite** (Компоновщик)
4. **Decorator** (Декоратор)
5. **Facade** (Фасад)
6. **Flyweight** (Приспособленец)
7. **Proxy** (Заместитель)

Классификация паттернов

Поведенческие

1. **Chain of Responsibility** (Цепочка Обязанностей)
2. **Command** (Команда)
3. **Interpreter** (Интерпретатор)
4. **Iterator** (Итератор)
5. **Mediator** (Посредник)
6. **Memento** (Хранитель)
7. **Observer** (Наблюдатель)
8. **State** (Состояние)
9. **Strategy** (Стратегия)
10. **Template Method** (Шаблонный Метод)
11. **Visitor** (Посетитель)

В основу категоризации каталога паттернов легли три простейшие объектно-ориентированные техники:

- *техника использования объектов-фабрик, порождающих объекты-продукты,*
- *техника использования объекта-фасада,*
- *техника диспетчеризации.*

Классификация паттернов

		Цель		
		Порождающие	*Структурные*	*Поведенческие*
Применимость (уровень)	*К классам*	Фабричный метод	Адаптер (класса)	Интерпретатор Шаблонный метод
	К объектам	Абстрактная фабрика Одиночка Прототип Строитель	Адаптер (объекта) Декоратор Заместитель Компоновщик Мост Приспособленец Фасад	Итератор Команда Наблюдатель Посетитель Посредник Состояние Стратегия Хранитель Цепочка обязанностей

Классификация паттернов

Цель паттерна – показывает его назначение.

- Целью *порождающих* паттернов, является организация процесса создания объектов.
- Целью *структурных* паттернов, является составление правильно организованных структур из объектов и классов.
- Целью *поведенческих* паттернов, является организация устойчивого (робастного) взаимодействия между классами или объектами, через правильное формирование информационных потоков.

Уровень паттерна - показывает область применения паттерна: к классам или к объектам.

- Паттерны *уровня классов* описывают отношения между классами и их подклассами. Такие отношения выражаются при помощи статических связей отношений – *наследования и реализации*.
- Паттерны *уровня объектов* описывают взаимодействия между объектами. Такие отношения выражаются при помощи динамических связей отношений – *ассоциации, агрегации и композиции*.

Примеры порождающих паттернов

Фабрика - Продукт

Техника использования объекта-фабрики для *порождения* объектов-продуктов, положена в основу всех *порождающих* паттернов.

Методы, принадлежащие объекту-фабрике, которые порождают и возвращают объекты-продукты, принято называть *фабричными-методами* (или виртуальными конструкторами).

Порождающие паттерны проектирования, которые абстрагируют процесс создания объекта класса (*инстанцирования* класса), позволяют сделать систему независимой от способа создания, композиции и представления объектов.

***Абстрактная фабрика** и **Абстрактный продукт**.*

Паттерн применяется случаях:

- когда программа должна быть независимой от процесса и типов создаваемых новых объектов;
- когда необходимо создать семейства или группы взаимосвязанных объектов, исключая возможность одновременного использования объектов из разных этих наборов в одном контексте.

Достоинства шаблона:

- изолирует конкретные классы;
- упрощает замену семейств продуктов;
- гарантирует сочетаемость продуктов.

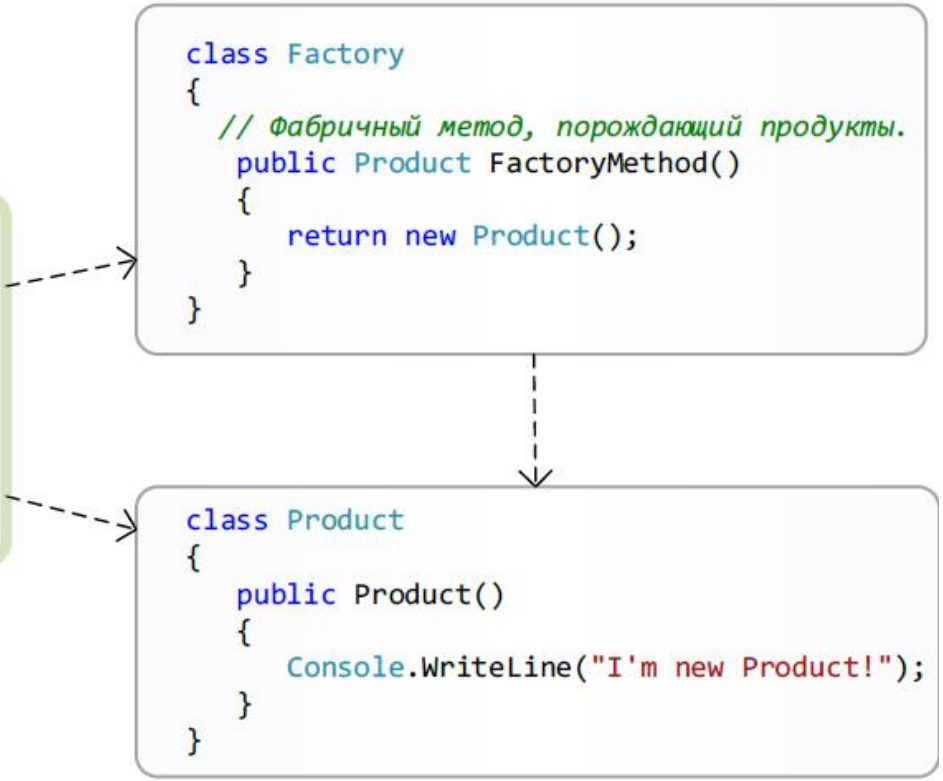
Примеры порождающих паттернов

```
class Program
{
    static void Main()
    {
        Factory factory = new Factory();

        Product product = factory.FactoryMethod();
    }
}
```

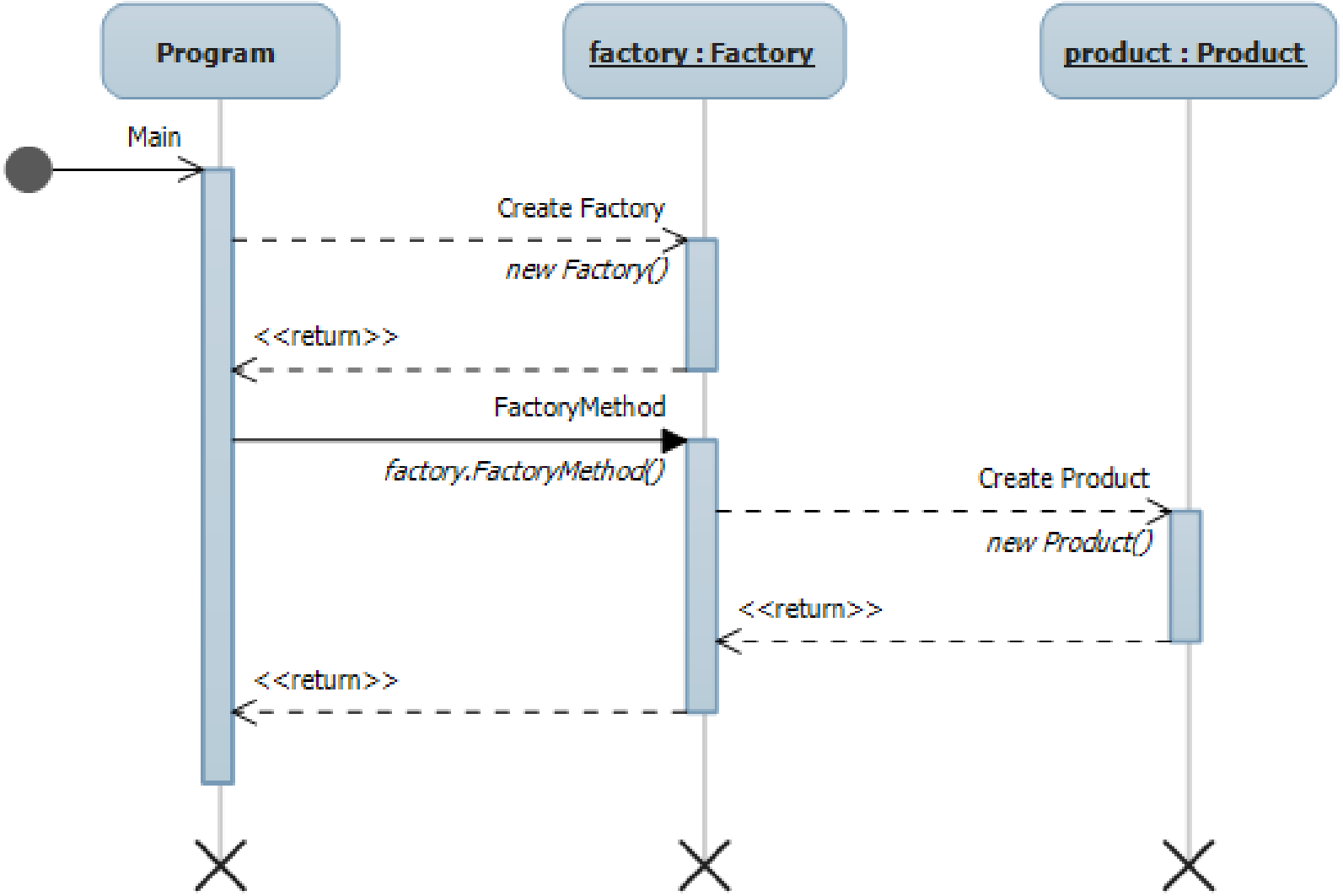
```
class Factory
{
  // Фабричный метод, порождающий продукты.
    public Product FactoryMethod()
    {
        return new Product();
    }
}
```

```
class Product
{
    public Product()
    {
        Console.WriteLine("I'm new Product!");
    }
}
```



Примеры порождающих паттернов

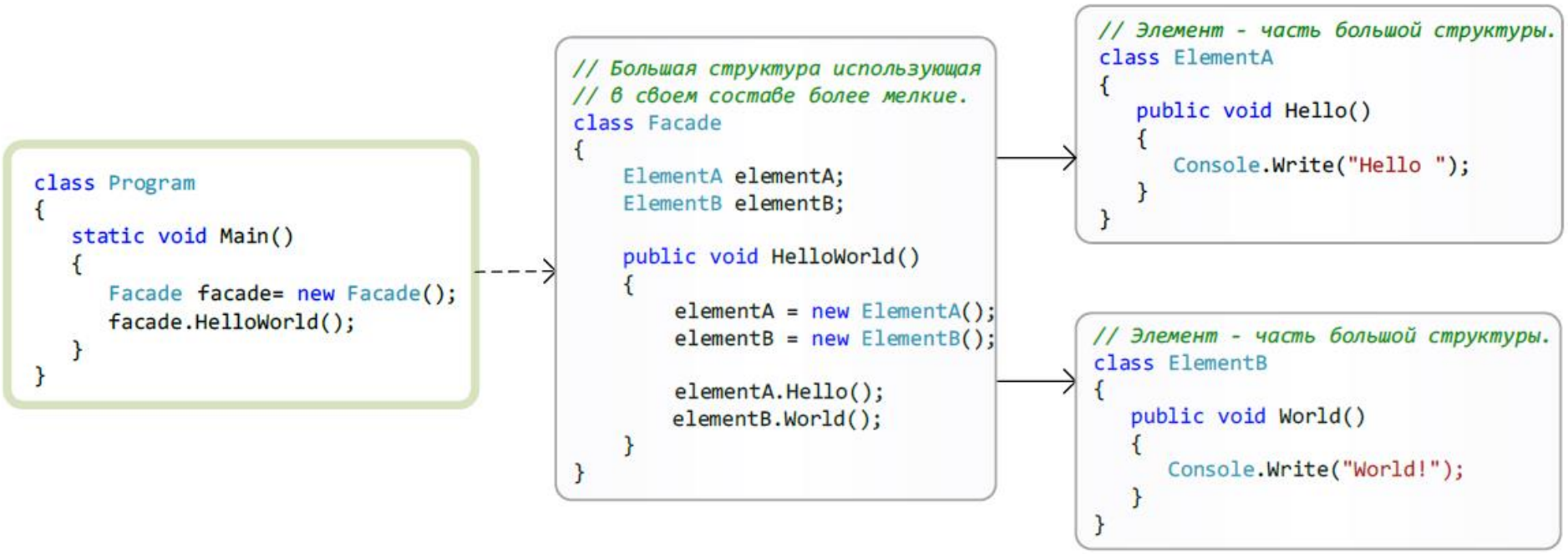
Диаграмма последовательностей фабричной техники



Примеры структурных паттернов

Структурные паттерны определяют различные сложные структуры, которые изменяют интерфейс уже существующих объектов или его реализацию, позволяя облегчить разработку и оптимизировать программу.

В основу ***структурных*** паттернов положена техника использования *объекта-фасада,* скрывающего за собой работу с неким подмножеством объектов.



```
class Program
{
   static void Main()
   {
      Facade facade= new Facade();
      facade.HelloWorld();
   }
}
```

```
// Большая структура использующая
// в своем составе более мелкие.
class Facade
{
    ElementA elementA;
    ElementB elementB;

    public void HelloWorld()
    {
        elementA = new ElementA();
        elementB = new ElementB();

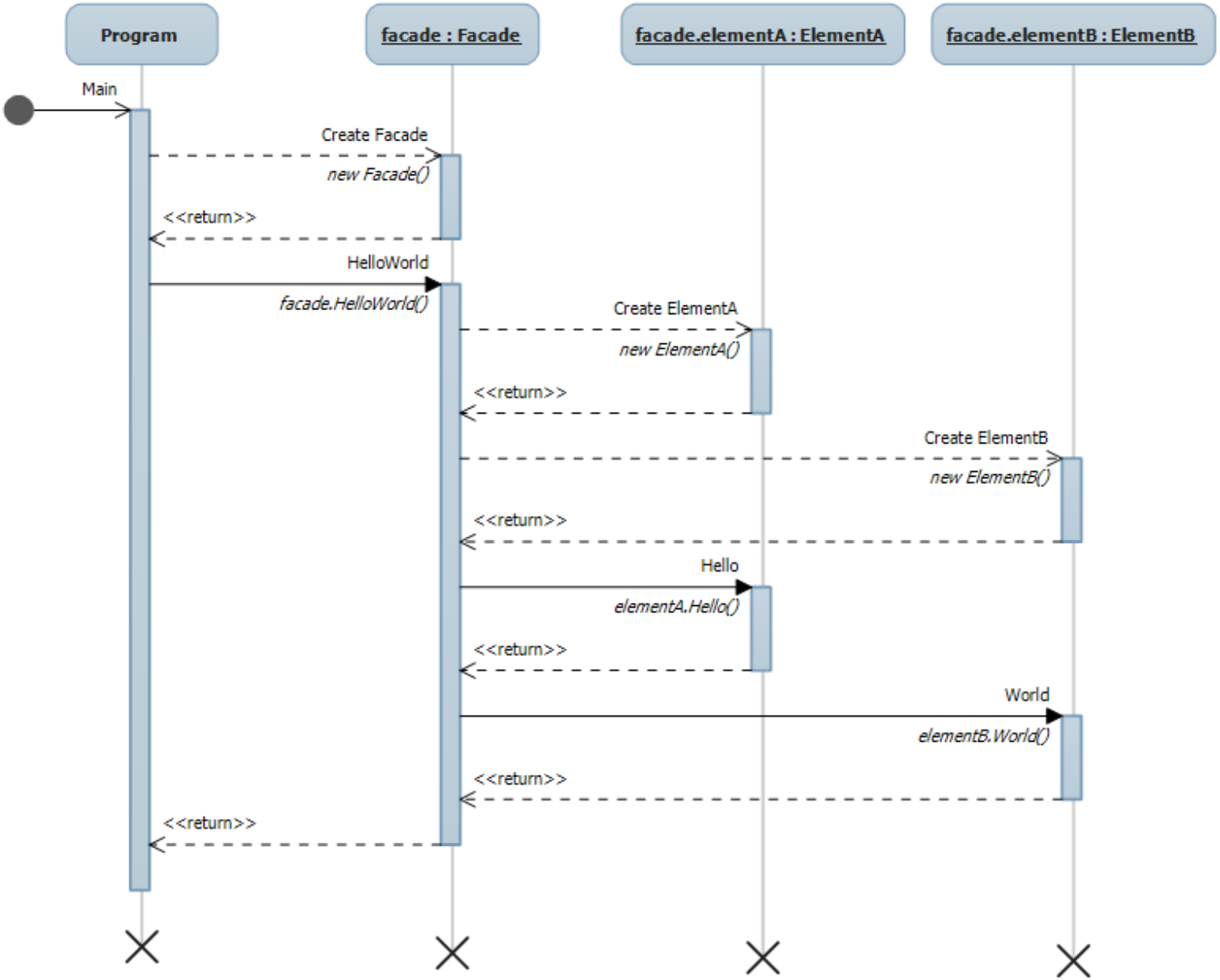
        elementA.Hello();
        elementB.World();
    }
}
```

```
// Элемент - часть большой структуры.
class ElementA
{
   public void Hello()
   {
      Console.Write("Hello ");
   }
}
```

```
// Элемент - часть большой структуры.
class ElementB
{
   public void World()
   {
      Console.Write("World!");
   }
}
```

Примеры порождающих паттернов

Диаграмма последовательностей фасадной техники



Примеры поведенческих паттернов

Диспетчеризация

Техника использования диспетчеризации имеет две формы: *«Цепочка объектов»* и *«Издатель- Подписчик»*. Эти техники были положены в основу поведенческих паттернов.

```
class Program
{
    static void Main()
    {
        Link3 chain = new Link3();
        chain.Method();
    }
}
```

```
class Link3
{
    public void Method()
    {
        Link2 link = new Link2();
        link.Method();
        Console.WriteLine("Link3");
    }
}
```

```
class Link2
{
    public void Method()
    {
        Link1 link = new Link1();
        link.Method();
        Console.WriteLine("Link2");
    }
}
```

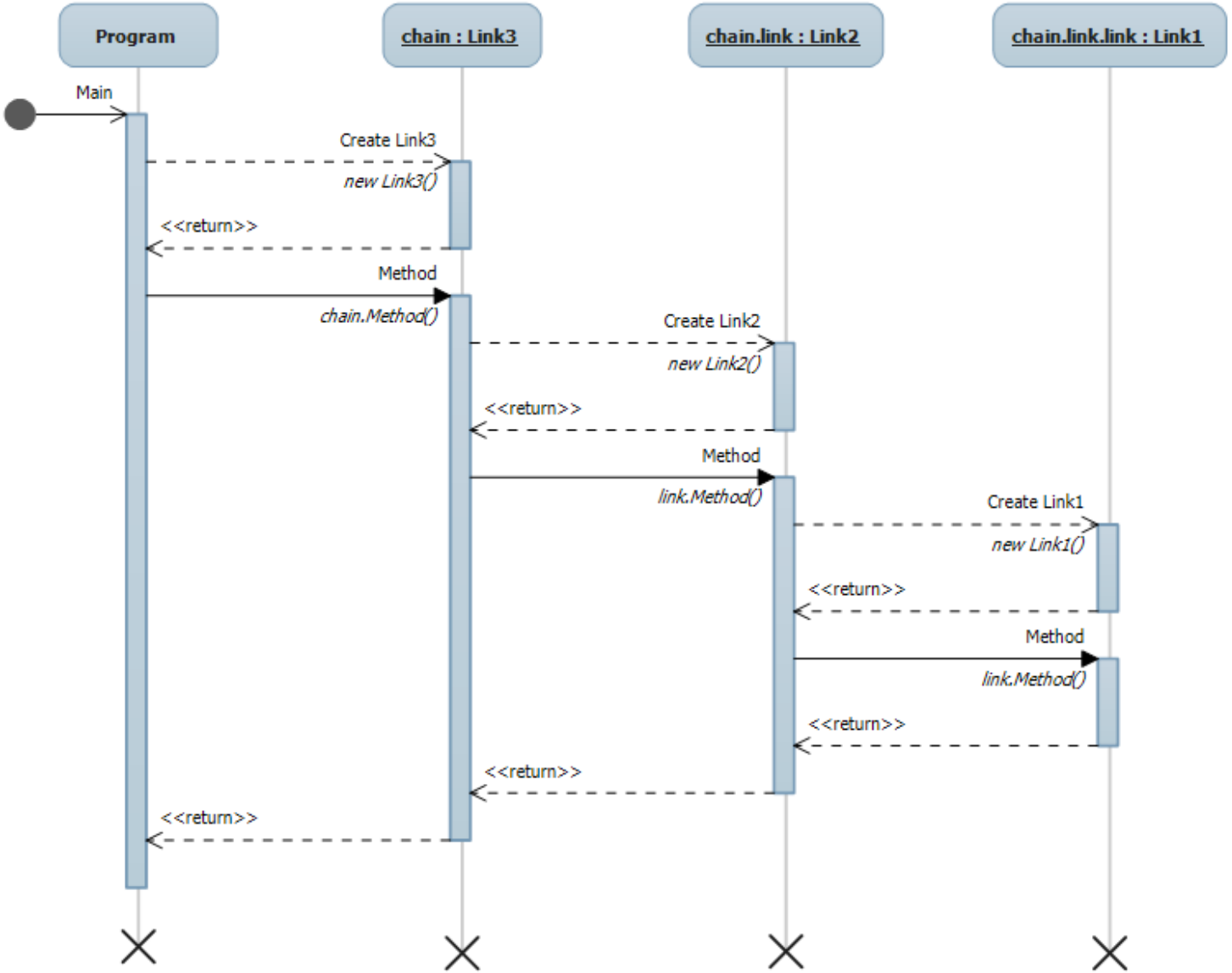
```
class Link1
{
    public void Method()
    {
        Console.WriteLine("Link1");
    }
}
```

Цепочка объектов

При использовании техники «Цепочка объектов» - объекты связываются в цепочку, вдоль которой происходит серия вызовов методов (посылка сообщений).

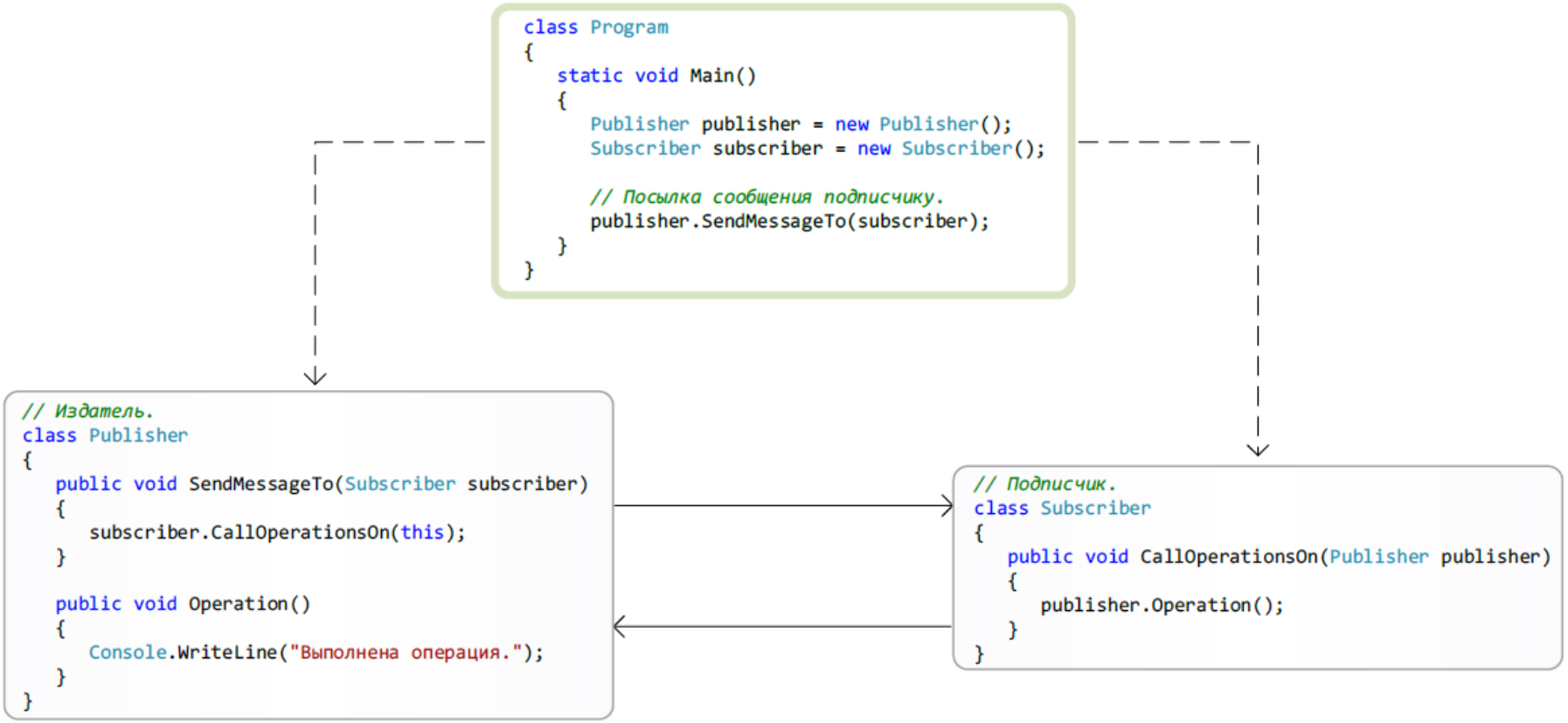
Примеры порождающих паттернов

Диаграмма последовательностей техники "Цепочка объектов».



Издатель-Подписчик

При использовании техники *«Издатель-Подписчик»* - объект-издатель вызывает метод на объекте-подписчике, а объект-подписчик после этого вызывает метод на объекте-издателе. Таким образом объект- издатель уведомляет объекта-подписчика о наступлении некоторого события.



```
class Program
{
   static void Main()
   {
      Publisher publisher = new Publisher();
      Subscriber subscriber = new Subscriber();

      // Посылка сообщения подписчику.
      publisher.SendMessageTo(subscriber);
   }
}
```

```
// Издатель.
class Publisher
{
   public void SendMessageTo(Subscriber subscriber)
   {
      subscriber.CallOperationsOn(this);
   }

   public void Operation()
   {
      Console.WriteLine("Выполнена операция.");
   }
}
```

```
// Подписчик.
class Subscriber
{
   public void CallOperationsOn(Publisher publisher)
   {
      publisher.Operation();
   }
}
```

Примеры порождающих паттернов

Диаграмма последовательностей техники "Издатель-подписчик"

